

Python Mini Project - Retail Sales Data Analysis & Visualization

Importing Required Libraries

```
In [56]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from datetime import datetime
import warnings
warnings.filterwarnings('ignore')
# Setting up Visualization Styles
plt.style.use('seaborn-v0_8')
sns.set_palette("husl")
print("All Libraries imported Succesfully")
```

All Libraries imported Succesfully

Load DataSet

```
In [57]: df = pd.read_csv('retail_sales.csv')
df.head(10)
```

```
Out[57]:
```

	Date	Category	Sales	Quantity	Pr
0	1 / 1 / 2 0 2 3	Electronics	1 1 4 9.0 1 4 2 4 6	1 1.0 3 8 3.6 6 4 2 4	
1	1 / 1 / 2 0 2 3	Clothing	9 5 8.5 2 0 7 1 0	7.0 2 2 4.0 5 4 0 4	
2	1 / 1 / 2 0 2 3	Home Goods	1 4 7 3.7 6 3 8 4 5	2.0 4 6 6.5 9 3 0 9	
3	1 / 1 / 2 0 2 3	Sports	1 2 3 0.2 3 0 4 1 9	6.0 1 2 3.3 1 0 4 6	
4	1 / 1 / 2 0 2 3	NaN?	8 2 8.5 8 5 9 5 0	1 2.0 8 8.5 9 1 3 5	
5	1 / 2 / 2 0 2 3	Electronics	7 2 2.7 7 5 1 4 9	1 0.0 8 2.3 9 6 1 4	
6	1 / 2 / 2 0 2 3	Clothing	7 8 8.9 9 6 8 5 9	1 5.0 2 2 5.2 7 1 0 7	
7	1 / 2 / 2 0 2 3	Home Goods	3 5 8.1 1 3 8 0 3	3.0 1 2 8.1 9 8 3 3	
8	1 / 2 / 2 0 2 3	Sports	8 1 9.9 2 3 8 4 5	2.0 2 5 0.2 9 8 0 7	
9	1 / 2 / 2 0 2 3	Books	1 2 8 4.2 3 1 9 4 6	7.0 3 6 3.4 3 6 3 5	

Basic Data Information And Structure

```
In [58]: print("===Data Set Basic Information===")
print(f"Dataset Dimensions: {df.shape[0]} rows x {df.shape[1]} columns")
print("\nColumns Name And DataType:")
print(df.dtypes)
print("\nMissing Values:")
print(df.isnull().sum())
print("\n DataSet Summry")
df.info()
```

```
===Data Set Basic Information===
Dataset Dimensions: 1825 rows x 6 columns
```

Columns Name And DataType:

```
Date          str
Category       str
Sales          float64
Quantity       float64
Profit         float64
Region         str
dtype: object
```

Missing Values:

```
Date          0
Category       4
Sales          2
Quantity       5
Profit         0
Region         5
dtype: int64
```

DataSet Summry

```
<class 'pandas.DataFrame'>
RangeIndex: 1825 entries, 0 to 1824
Data columns (total 6 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Date        1825 non-null   str
1   Category    1821 non-null   str
2   Sales       1823 non-null   float64
3   Quantity    1820 non-null   float64
4   Profit      1825 non-null   float64
5   Region      1820 non-null   str
dtypes: float64(3), str(3)
memory usage: 124.3 KB
```

Data Type Conversion And Cleaning

```
In [59]: print("=== DATA CLEANING AND TYPE CONVERSION ===")

# Convert DATE to DATETIME Format

df['Date']=pd.to_datetime(df['Date'])
```

```
#Extract Additional Time Feature
df['Month']=df['Date'].dt.month
df['Quarter']=df['Date'].dt.quarter
df['DayOfWeek']=df['Date'].dt.day_name()
df['MonthName']=df['Date'].dt.month_name()

# Extract Numeric Columns Are Properly Formatted
df.columns = df.columns.str.strip()

numeric_columns = ['Sales', 'Quantity', 'Profit']
for col in numeric_columns:
    df[col] = pd.to_numeric(df[col], errors='coerce')
print("Data Type After Conversion:")
print(df.dtypes)
print(f"\n Date range: {df['Date'].min()} to {df['Date'].max()}")
```

=== DATA CLEANING AND TYPE CONVERSION ===

Data Type After Conversion:

```
Date          datetime64[us]
Category      str
Sales          float64
Quantity       float64
Profit         float64
Region         str
Month          int32
Quarter        int32
DayOfWeek      str
MonthName      str
dtype: object
```

Date range: 2023-01-01 00:00:00 to 2023-12-31 00:00:00

Basic Statistical Summary

```
In [60]: print("=== STATISTICAL SUMMARY===")
print("Decriptive Statistics For Numerical Columns:")
print(df[['Sales','Quantity','Profit']].describe())
```

=== STATISTICAL SUMMARY===

Decriptive Statistics For Numerical Columns:

	Sales	Quantity	Profit
count	1823.000000	1820.000000	1825.000000
mean	980.482677	10.053846	248.696130
std	336.756618	5.515370	118.159627
min	0.000000	0.000000	0.000000
25%	784.071354	5.000000	156.916498
50%	997.725134	10.000000	229.344557
75%	1208.512721	15.000000	324.867847
max	1888.932537	19.000000	703.228418

Category-Wise Summary

```
In [61]: print("\nCategory-wise Summary:")
category_summary = df.groupby('Category').agg({
    'Sales': ['count', 'sum', 'mean', 'std'],
    'Profit': ['sum', 'mean'],
```

```
'Quantity': ['sum', 'mean']
}).round(2)
print(category_summary)
```

Category-wise Summary:

	Sales				Profit		Quantity	
	count	sum	mean	std	sum	mean	sum	\
Category								
Books	364	370971.87	1019.15	336.39	92304.45	253.58	3737.0	
Clothing	365	350981.82	961.59	337.49	90082.12	246.80	3641.0	
Electronics	360	364090.41	1011.36	344.25	90439.70	250.53	3595.0	
Home Goods	364	347493.83	954.65	332.39	87447.82	240.24	3777.0	
NaN?	1	828.59	828.59	NaN	88.59	88.59	12.0	
Nan	1	823.19	823.19	NaN	253.43	253.43	4.0	
Null	1	1106.85	1106.85	NaN	381.78	381.78	11.0	
Sports	363	346685.06	955.06	331.20	92065.22	252.93	3492.0	

	mean
Category	
Books	10.27
Clothing	9.98
Electronics	9.96
Home Goods	10.38
NaN?	12.00
Nan	4.00
Null	11.00
Sports	9.73

Region-Wise Summary

```
In [62]: print("\nRegion-wise Summary")
region_summary = df.groupby('Region').agg({
    'Sales': ['sum', 'mean'],
    'Profit': ['sum', 'mean']
}).round(2)
print(region_summary)
```

Region-wise Summary

	Sales		Profit	
	sum	mean	sum	mean
Region				
East	393441.61	991.04	98973.62	248.68
Nan	1716.71	1716.71	545.62	545.62
North	448295.03	976.68	112126.47	243.75
South	469471.28	969.98	118036.97	243.88
West	470514.37	986.40	122517.78	256.85

Monthly Sales Trend Analysis

```
In [63]: print("=== MONTHLY SALES TREND ANALYSIS ===")
# Calculate monthly sales
monthly_sales = df.groupby(['Month', 'MonthName']).agg({
    'Sales': 'sum',
    'Profit': 'sum',
    'Quantity': 'sum'
})
```

```

}).reset_index()

monthly_sales = monthly_sales.sort_values('Month')
print("Monthly Sales Summary")
print(monthly_sales)

```

=== MONTHLY SALES TREND ANALYSIS ===

Monthly Sales Summary

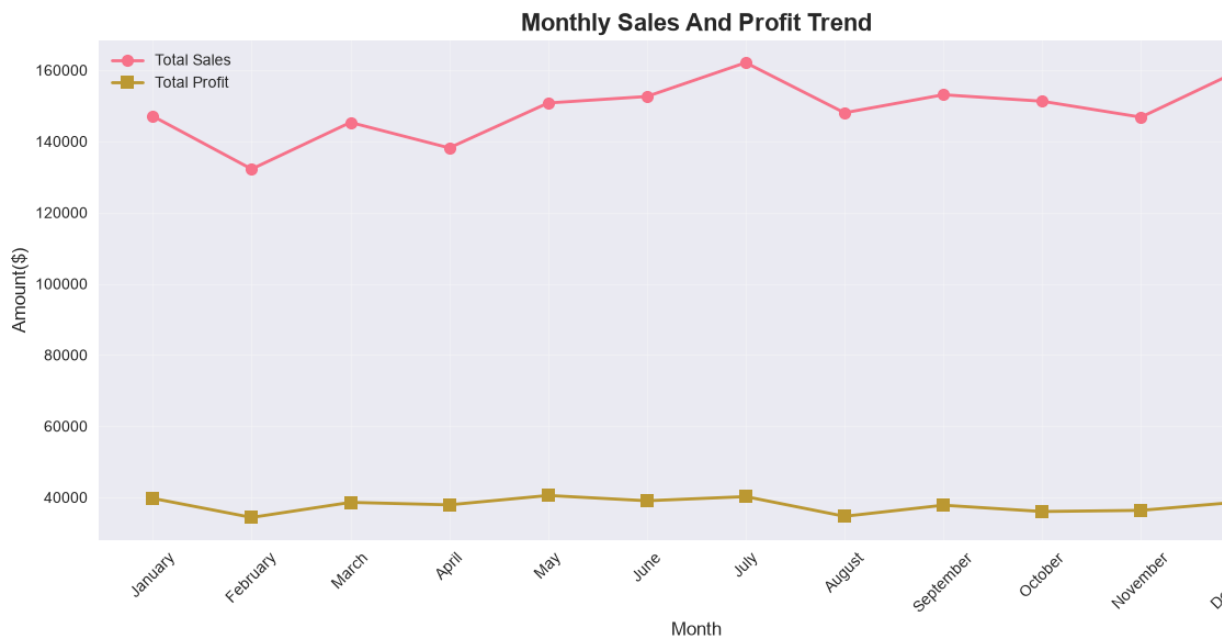
	Month	MonthName	Sales	Profit	Quantity
0	1	January	147068.619477	39711.436221	1486.0
1	2	February	132215.411776	34353.123612	1517.0
2	3	March	145321.200626	38584.599285	1447.0
3	4	April	138160.991209	37895.203187	1446.0
4	5	May	150774.977103	40527.689566	1603.0
5	6	June	152605.749860	39042.686605	1570.0
6	7	July	162111.115687	40208.956109	1536.0
7	8	August	148020.452251	34675.627875	1489.0
8	9	September	153094.700793	37796.761379	1598.0
9	10	October	151272.280840	36019.384158	1569.0
10	11	November	146795.613628	36344.048332	1461.0
11	12	December	159978.807192	38710.920857	1576.0

Visualize Monthly Sales Trend

```

In [64]: plt.figure(figsize=(12, 6))
plt.plot(monthly_sales['MonthName'], monthly_sales['Sales'],
         marker='o', linewidth=2, markersize=8, label='Total Sales')
plt.plot(monthly_sales['MonthName'], monthly_sales['Profit'],
         marker='s', linewidth=2, markersize=8, label='Total Profit')
plt.title('Monthly Sales And Profit Trend', fontsize=16, fontweight='bold')
plt.xlabel('Month', fontsize=12)
plt.ylabel('Amount($)', fontsize=12)
plt.legend()
plt.grid(True, alpha=0.3)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

```



Category Performance Analysis

```
In [65]: print("=== CATEGORY PERFORMANCE ANALYSIS===")
category_performance = df.groupby('Category').agg({
    'Sales': 'sum',
    'Profit': 'sum',
    'Quantity': 'sum'
}).sort_values('Sales', ascending=False)
print("Category Performance (Sorted By Sales):")
print(category_performance)
```

```
=== CATEGORY PERFORMANCE ANALYSIS===
Category Performance (Sorted By Sales):
```

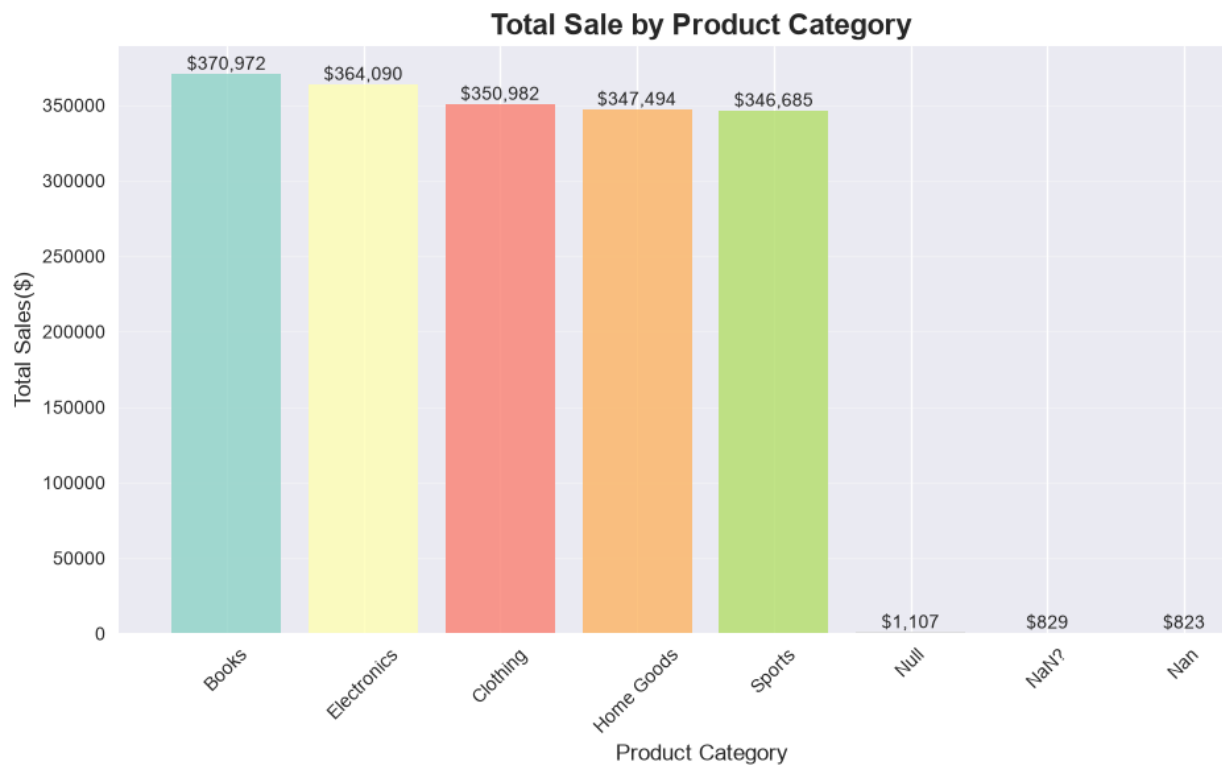
	Sales	Profit	Quantity
Category			
Books	370971.872558	92304.452141	3737.0
Electronics	364090.406288	90439.697795	3595.0
Clothing	350981.822773	90082.119920	3641.0
Home Goods	347493.827088	87447.820056	3777.0
Sports	346685.063298	92065.220851	3492.0
Null	1106.847949	381.784578	11.0
NaN?	828.585950	88.591355	12.0
Nan	823.190573	253.433301	4.0

Visualize Top Categories By Sales

```
In [66]: plt.figure(figsize=(10,6))
colors = plt.cm.Set3(np.linspace(0,1,len(category_performance)))
bars = plt.bar(category_performance.index, category_performance['Sales'],
               color=colors,alpha=0.8)

#Add Value Labels on Bars
for bar in bars:
    height = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2., height,
             f'${height:,.0f}', ha='center', va='bottom')

plt.title('Total Sale by Product Category ', fontsize=16, fontweight='bold')
plt.xlabel('Product Category ', fontsize=12)
plt.ylabel('Total Sales($)', fontsize=12)
plt.xticks(rotation=45)
plt.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.show()
```



Regional Performance Analysis

```
In [67]: print("=== REGIONAL PERFORMANCE ANALYSIS===")

regional_performance = df.groupby('Region').agg({
    'Sales': ['sum', 'mean', 'count'],
    'Profit': ['sum', 'mean']
}).round(2)
print("Regional Performance :")
print(regional_performance)
```

=== REGIONAL PERFORMANCE ANALYSIS===

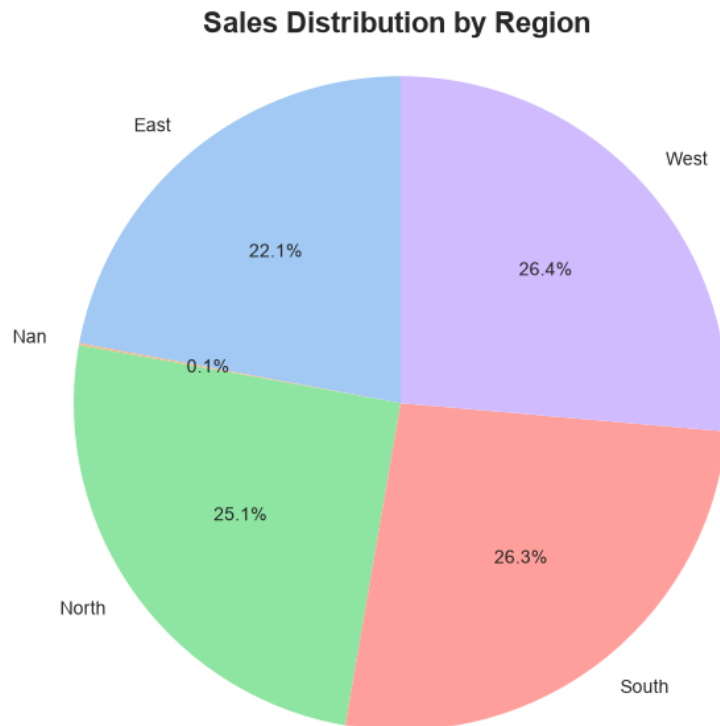
Regional Performance :

Region	Sales			Profit	
	sum	mean	count	sum	mean
East	393441.61	991.04	397	98973.62	248.68
Nan	1716.71	1716.71	1	545.62	545.62
North	448295.03	976.68	459	112126.47	243.75
South	469471.28	969.98	484	118036.97	243.88
West	470514.37	986.40	477	122517.78	256.85

Visualize Regional Sales Distribution

```
In [68]: plt.figure(figsize=(10,6))
regional_data = df.groupby('Region')['Sales'].sum()
plt.pie(regional_data, labels=regional_data.index, autopct='%1.1f%%',
        startangle=90, colors=sns.color_palette("pastel"))
plt.title('Sales Distribution by Region ', fontsize=16, fontweight='bold')
plt.axis('equal')
```

```
plt.tight_layout()
plt.show()
```



Day Of Week Analysis

```
In [69]: print("=== DAY OF WEEK ANALYSIS ===")
# define proper order for days
day_order = ['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday']

# 'DayOfWeek' column ko properly categorical convert kiya (string literal)
df['DayOfWeek'] = pd.Categorical(df['DayOfWeek'], categories=day_order, ordered=True)

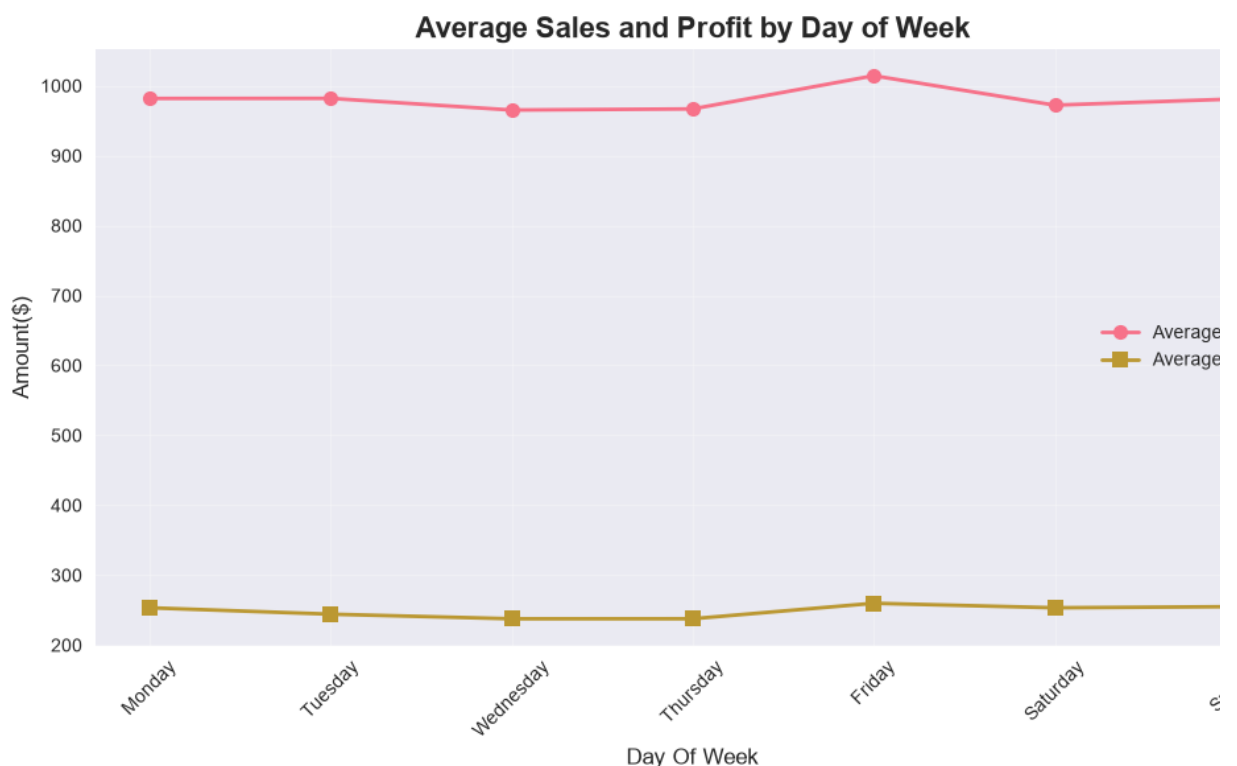
daily_performance = df.groupby('DayOfWeek', observed=False).agg({
    'Sales': 'mean',
    'Profit': 'mean',
    'Quantity': 'mean'
}).round(2)

print("Average Performance by Day Of Week:")
print(daily_performance)
```

```
=== DAY OF WEEK ANALYSIS ===
Average Performance by Day Of Week:
      Sales  Profit  Quantity
DayOfWeek
Monday    981.77   253.32    10.24
Tuesday   981.92   244.18    10.31
Wednesday 965.23   237.57    10.18
Thursday  966.83   237.62     9.78
Friday   1014.31   259.80    10.00
Saturday  972.20   253.32     9.64
Sunday    981.11   254.94    10.22
```

Visualize Daily Sales Pattern

```
In [70]: plt.figure(figsize=(10,6))
plt.plot(daily_performance.index, daily_performance['Sales'],
         marker='o', linewidth=2, markersize=8, label='Average Sales')
plt.plot(daily_performance.index, daily_performance['Profit'],
         marker='s', linewidth=2, markersize=8, label='Average Profit')
plt.title("Average Sales and Profit by Day of Week", fontsize=16, fontwei
plt.xlabel('Day Of Week', fontsize=12)
plt.ylabel('Amount($)', fontsize=12)
plt.legend()
plt.grid(True, alpha=0.3)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



Correlation Analysis

```
In [71]: print("=== CORRELATION ANALYSIS ===")

# Select Numerical Columns For correlation
numerical_df = df[['Sales', 'Quantity', 'Profit', 'Month']]
correlation_matrix = numerical_df.corr()

print("Correlation Matrix")
print(correlation_matrix)
```

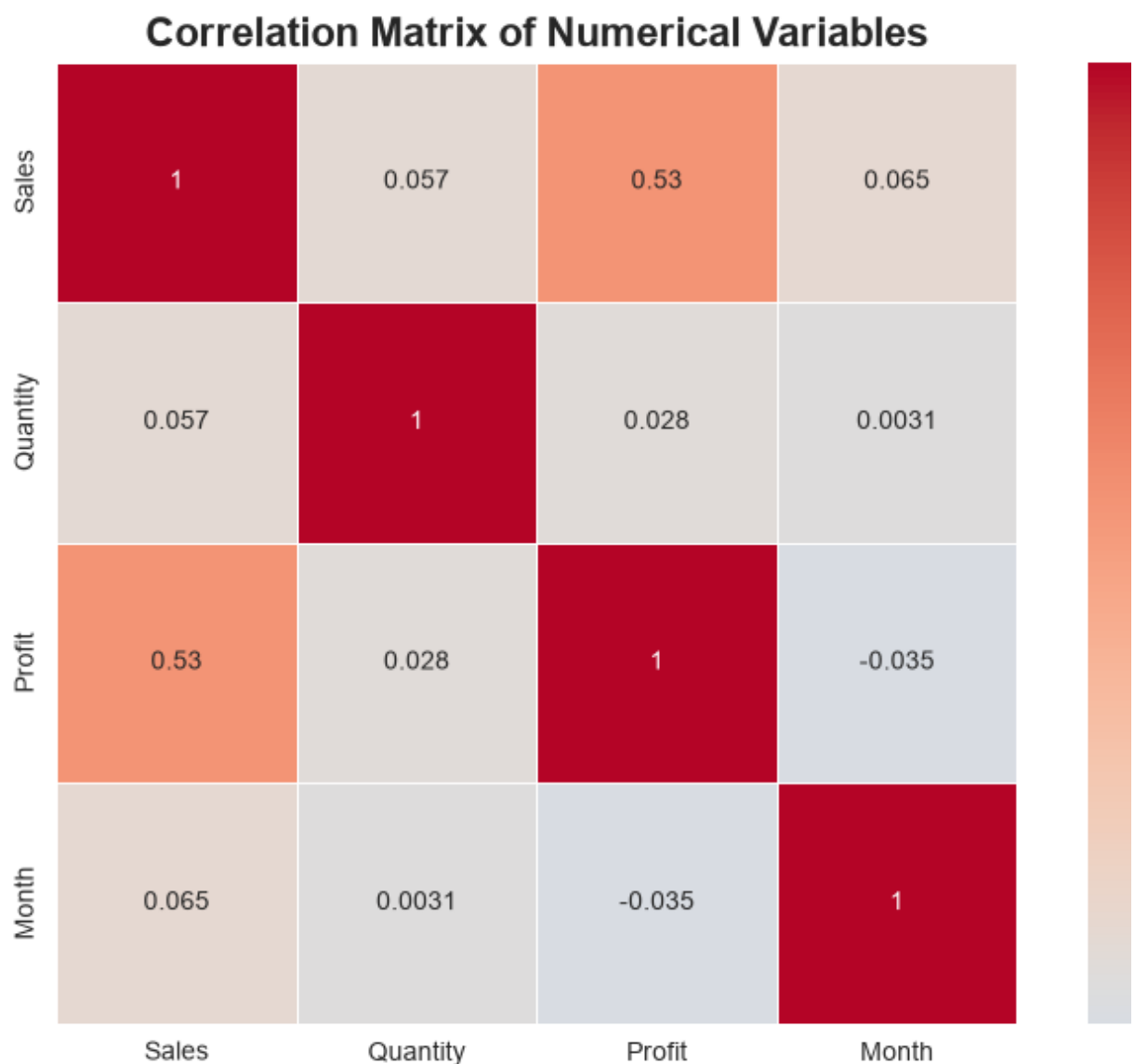
=== CORRELATION ANALYSIS ===

Correlation Matrix

	Sales	Quantity	Profit	Month
Sales	1.000000	0.056824	0.525041	0.065291
Quantity	0.056824	1.000000	0.027875	0.003127
Profit	0.525041	0.027875	1.000000	-0.034678
Month	0.065291	0.003127	-0.034678	1.000000

Heatmap Visualization For Correlation

```
In [72]: plt.figure(figsize=(8, 6))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', center=0,
            square=True, linewidths=0.5)
plt.title('Correlation Matrix of Numerical Variables', fontsize=16, fontw
plt.tight_layout()
plt.show()
```



Mini Dashboard 2 x 2

```
In [73]:
```

```

# Mini Dashboard

print("=== RETAIL SALES DASHBOARD ===")

# Create a 2x2 dashboard
fig, axes = plt.subplots(2, 2, figsize=(15, 12))

fig.suptitle(
    "Retail Sales Analysis Dashboard",
    fontsize=20,
    fontweight='bold'
)

# Chart 1: Monthly Sales & Profit Trend

axes[0, 0].plot(
    monthly_sales['MonthName'],
    monthly_sales['Sales'],
    marker='o',
    linewidth=2,
    label='Sales'
)

axes[0, 0].plot(
    monthly_sales['MonthName'],
    monthly_sales['Profit'],
    marker='s',
    linewidth=2,
    label='Profit'
)

axes[0, 0].set_title(
    'Monthly Sales & Profit Trend',
    fontweight='bold'
)

axes[0, 0].set_xlabel('Month')
axes[0, 0].set_ylabel('Amount ($)')
axes[0, 0].legend()
axes[0, 0].grid(True, alpha=0.3)
axes[0, 0].tick_params(axis='x', rotation=45)

# Chart 2: Sales by Product Category

category_data = df.groupby('Category')['Sales'].sum().sort_values(
    ascending=False
)

axes[0, 1].bar(
    category_data.index,
    category_data.values
)

axes[0, 1].set_title(
    'Total Sales by Product Category',
    fontweight='bold'
)

```

```

axes[0, 1].set_xlabel('Product Category')
axes[0, 1].set_ylabel('Total Sales ($)')
axes[0, 1].tick_params(axis='x', rotation=45)
axes[0, 1].grid(axis='y', alpha=0.3)

# Chart 3: Sales Distribution by Region

regional_data = df.groupby('Region')['Sales'].sum()

axes[1, 0].pie(
    regional_data,
    labels=regional_data.index,
    autopct='%1.1f%%',
    startangle=90
)

axes[1, 0].set_title(
    'Sales Distribution by Region',
    fontweight='bold'
)

# Chart 4: Average Sales & Profit by Day

axes[1, 1].plot(
    daily_performance.index,
    daily_performance['Sales'],
    marker='o',
    linewidth=2,
    label='Average Sales'
)

axes[1, 1].plot(
    daily_performance.index,
    daily_performance['Profit'],
    marker='s',
    linewidth=2,
    label='Average Profit'
)

axes[1, 1].set_title(
    'Average Sales & Profit by Day',
    fontweight='bold'
)

axes[1, 1].set_xlabel('Day of Week')
axes[1, 1].set_ylabel('Amount ($)')
axes[1, 1].legend()
axes[1, 1].grid(True, alpha=0.3)
axes[1, 1].tick_params(axis='x', rotation=45)

# Adjust Dashboard Layout

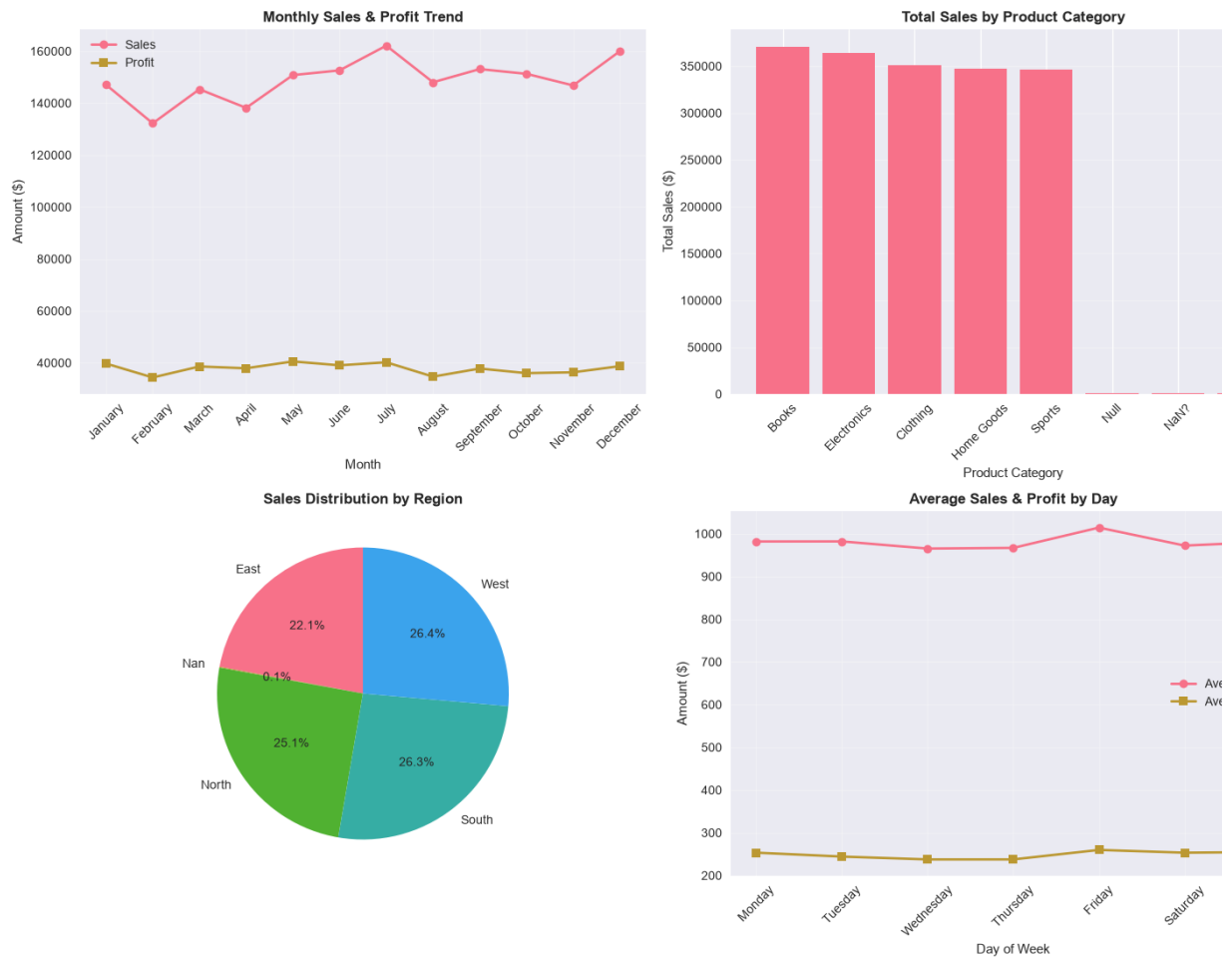
plt.tight_layout(rect=[0, 0, 1, 0.96])

plt.show()

```

=== RETAIL SALES DASHBOARD ===

Retail Sales Analysis Dashboard



Key Insights and Business Recommendations

```
In [74]: print("=== KEY INSIGHTS AND RECOMMENDATION===")
print("\n📊 KEY FINDINGS:")
print("1. Top Performance Category:", category_performance.index[0])
print("2. Best Performing Region:", regional_data.idxmax())
print("3. Highest Sales Month:", monthly_sales.loc[monthly_sales['Sales'].idxmax()])
print("4. Best Day of Week:", daily_performance['Sales'].idxmax())
print("5. Total Annual Sales: ${:,.2f}".format(df['Sales'].sum()))
print("6. Total Annual Profit: ${:,.2f}".format(df['Profit'].sum()))

# Business Recommendation
print("\n💡 Business Recommendations")
print("1. Focus Marketing efforts On", category_performance.index[0], "category")
print("2. Investigate Performance in Lower-performing Regions")
print("3. Plan Promotions Around", daily_performance['Sales'].idxmax(), "day")
print("4. Allocate Inventory Based On Monthly Trend Patterns")
print("5. Monitor Correlation Between Quantity And Profit For Pricing Strategy")

print("\n📈 PROFITABILITY ANALYSIS:")
profit_margin = (df['Profit'].sum() / df['Sales'].sum()) * 100
print(f"Overall Profit Margin: {profit_margin:.2f}%")

# Category Profit Margins
category_margins = (category_performance['Profit'] / category_performance['Sales']) * 100
print("\nProfit Margins By Category:")
```

```
for category, margin in category_margins.items():  
    print(f" {category}: {margin:.2f}%")
```

=== KEY INSIGHTS AND RECOMMENDATION===

KEY FINDINGS:

1. Top Performance Category: Books
2. Best Performing Region: West
3. Highest Sales Month: July
4. Best Day of Week: Friday
5. Total Annual Sales: \$1,787,419.92
6. Total Annual Profit: \$453,870.44

Business Recommendations

1. Focus Marketing efforts On Books category
2. Investigate Performance in Lower-performing Regions
3. Plan Promotions Around Friday To Maximize Sales
4. Allocate Inventory Based On Monthly Trend Patterns
5. Monitor Correlation Between Quantity And Profit For Pricing Strategy

PROFITABILITY ANALYSIS:

Overall Profit Margin: 25.39%

Profit Margins By Category:

Books: 24.88%
Electronics: 24.84%
Clothing: 25.67%
Home Goods: 25.17%
Sports: 26.56%
Null: 34.49%
NaN?: 10.69%
Nan: 30.79%

In []: